

P4033R0

Synthesizing enum at compile time with `define_enum`

Published Proposal, 2026-03-30

This version:

<https://wg21.link/P4033R0>

Author:

Aurelien Cassagnes

Audience:

SG7

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:

C++29

Table of Contents

1 Motivation

2 What about unscoped enum ?

3 Feature

3.1 `enumerator_options`

3.2 `enumerator_spec`

3.3 `is_enumerator_spec`

3.4 `define_enum`

4 Wording

4.1 Library

4.1.1 Meta synopsis [meta.syn]

4.1.2 Reflection union definition generation [meta.reflection.define.enum]

5 Status

6 Acknowledgements

References

Normative References

§ 1. Motivation

The generative capability of reflection as they were introduced in C++26 are limited to `define_aggregate`, with no quick paths to more powerful facilities (See [p3294r2] for example). We can already leverage `define_aggregate` to impressive effects (See a JSON parser [here](#)), here we introduce another basic and lightweight building block: `define_enum`.

enum-based variant

Using `define_enum()` we can simulate some enum-based discriminant `variant`

```
template<class... Ts>
struct named_variant_T {
    std::variant<Ts...> v;

    enum class kind : std::size_t;
    constexpr {
        std::vector<std::meta::info> enumerators;
        for (auto type : {^Ts...}) {
            enumerators.push_back(enumerator_spec({
                .name = std::define_static_string(identifier_of(type))
            }));
        }
        enumerators.push_back(enumerator_spec({.name = "Default"}));
        define_enum(^kind, enumerators);
    }

    constexpr kind which() const noexcept {
        auto i = v.index();
        if (i == std::variant_npos) return kind::Default;
        return static_cast<kind>(i);
    }
    // ...
};

using Message = named_variant_T<X, Y, Z>;
```

This unlocks a `variant` that has some favorable properties
Looking at a simple example of non-`visit()` based access

Before	After
<pre>using VarPet = std::variant<Dog, Tanuki, Cat>; // To keep in sync enum VariantPetIndex { Dog, Tanuki, Cat, }; //... VarPet animal(Dog{}); // ... switch (animal.index()) { default: break; case VariantPetIndex::Dogs break; case VariantPetIndex::Tanuki: break; case VariantPetIndex::Cats break; }</pre>	<pre>using VarPet = named_variant_T<Dog, Tanuki, Ca //... VarPet animal(Dog{}); switch (animal.which()) { case VarPet::kind::Default: break; case VarPet::kind::Dog: break; case VarPet::kind::Tanuki: break; case VarPet::kind::Cats break; }</pre>

If `variant<Dog, Tanuki, Cat>` is later augmented with `variant<Dog, Tanuki, Racoon, Cat>`, or the variant are shuffled, the index-based switch table will break silently, not the enum based one. Now, to be fair, `visit()` does not suffer this limitation. However we should recognize that

1. It is considerably more verbose
2. switch tables lend themselves easily to jump table dispatch (which is fast, which is nice)

See [this example](#) where we also leverage annotations on enumerators.

§ 2. What about unscoped enum ?

Note that above, we made the explicit choice of targeting scoped enumeration only, and so the following is not valid

```
enum HealthySnacks: int;
constexpr {
    define_enum(^^HealthySnacks, {
        enumerator_spec({.name = "Carrot"}),
        enumerator_spec({.name = "Celeri"}),
        enumerator_spec({.name = "BabaAuRhum"})
    });
}

// From here on out BabaAuRhum == 2
```

Our rationale behind this conservative approach is entirely rooted in our implementation experience (granted not extensive). Synthesizing enumerators of a scoped enum is a fairly simple operation, on the other hand rewiring the proper context for enumerators of an unscoped enum is quite more troublesome and error prone... Also having constants popping out of existing in a fairly large scope could be troublesome... Hence, for now, we made the cautious choice of limiting the operation to scoped enum only.

§ 3. Feature

The actual feature proposed here is `define_enum()`, allowing to complete an opaque scoped enumeration alongside the description of its enumerators. In turn, `define_enum` relies on a lightweight description of the enumerators (enumerator options), that are passed to `enumerator_spec()`.

All those pieces will be described here, ultimately it should feel familiar to any `define_aggregate()` enthusiast.

§ 3.1. enumerator_options

```
template<integral I>
struct enumerator_options {
    string      name;
    optional<I> value      = {};
    vector<info> annotations = {};
    vector<info> attributes  = {};
};
```

As when defining manually an enumerator, the integral `value` is optional. If omitted it will be computed in the same fashion that is done already (incrementing previous value). Diverging with the original design of `define_aggregate()`, annotations and attributes are directly supported here via `.annotations` and `.attributes`.

Finally note that the support for `.attributes` here relies entirely on the adoption of attributes reflection via [\[p3385r7\]](#).

§ 3.2. `enumerator_spec`

```
constexpr info enumerator_spec(enum_options props);
```

`enumerator_spec` returns the reflection of an enumerator description from the passed in properties.

§ 3.3. `is_enumerator_spec`

```
constexpr bool is_enumerator_spec(info r);
```

`is_enumerator_spec` returns `true` when `r` is a reflection returned by `enumerator_spec`. It is kept distinct from the reflection of an enumerator, since, while similar on some front they are used for distinct purposes.

§ 3.4. `define_enum`

```
template <reflection_range R = initializer_list<info>>  
constexpr info define_enum(info targetEnum, R&& members);
```

Finally `define_enum` completes a scoped enumeration declaring a set of enumerators under it. We pass in the reflection of the opaque scoped enumeration we want to complete, and a sequence of reflection obtained via `enumerator_spec`. As was the decision for `define_aggregate()`, when it comes to running compile time operations with side effect, we force `define_enum` to appear within a `constexpr` block. Only following the block, is the enumeration completed with its enumerators as specified.

§ 4. Wording

§ 4.1. Library

§ 4.1.1. Meta synopsis [meta.syn]

Add after [meta.reflection.define.aggregate]

```
namespace std {  
    // ...  
  
    // [meta.reflection.define.enum], enum definition generation  
  
    template<integral I> struct enum_options;  
  
    constexpr info enumerator_spec(enum_options options);  
  
    constexpr bool is_enumerator_spec(info r);  
  
    template <reflection_range R = initializer_list<info>>  
        constexpr info define_enum(info targetEnum, R&& members);
```

§ 4.1.2. Reflection union definition generation [meta.reflection.define.enum]

```
namespace std::meta {
    template<integral I>
    struct enumerator_options {
        optional<string> name = std::nullopt;
        optional<I> value = {};
        vector<info> attributes = {};
        vector<info> annotations = {};
    };
}
constexpr info enumerator_spec(enumerator_options options);
```

1 *Returns:* A reflection of an enumerator description (N, V, AT, AN) where

- (1.1) — N is either the identifier held by `options.name` or $\perp$ if `options.name` does not contain a value,
- (1.2) — V is either the value held by `options.value` or $\perp$ if `options.value` does not contain a value,
- (1.3) — AT is a potentially empty sequence of attribute reflections from `options.attributes`, and
- (1.4) — AN is a potentially empty sequence of values `constant_of(r)` for each r in `options.annotations`

Throws: `meta::exception` unless the following conditions are met:

- (1.1) — `options.name.has_value()` is false, or `name.value()` is a valid identifier ([lex.name]) that is not a keyword ([lex.key]), and
- (1.2) — for each r in `options.attributes`, `is_attribute(options.attributes_of[r])` is true, and
- (1.3) — for each r in `options.annotations`, `type_of(r)` represents a non-array object type, and evaluation of `constant_of(r)` does not exit via an exception.

```
constexpr bool is_enumerator_spec(info r);
```

1 *Returns:* true if r represents the reflection of an enumerator description. Otherwise, false

```
template <reflection_range R = initializer_list<info>>
constexpr info define_enum(info targetEnum, R&& members);
```

1 Let E be the enum represented by `targetEnum` and r_i be the i^{th} reflection value in `members`. For every r_i in `members`, let (N_i, V_i, AT_i, AN_i) be the corresponding enumerator description reflection.

Constant when:

- (1.1) — `is_enumerator_spec(ri)` is true, and
- (1.2) — `targetEnum` is a reflection that represents a scoped enumeration type, and
- (1.3) — `targetEnum` is an opaque enumeration from every point in the evaluation context, and
- (1.4) — for every pair (i, j) where $i < j$ and N_i is not $\perp$ and N_j is not $\perp$, then either:
 - (1.1) — N_i is not the same identifier as N_j or
 - (1.2) — N_i is the identifier `_` (U+005f low line).

Effects: Produces an injected declaration D ([expr.const]) that defines E and has properties as follows:

- (1.1) — The target scope of **D** is the scope to which **E** belongs ([basic.scope.scope]).
 - (1.2) — The locus of **D** follows immediately after the core constant expression currently under evaluation.
 - (1.3) — The injected definition has an *enumerator-list* with one *enumerator-definition* for each element of **members**, in order. For the i^{th} element of **members**, the corresponding *enumerator-definition*:
 - (1.1) — is preceded by the attributes denoted by the attribute reflections in AT_i , and
 - (1.2) — is preceded by an annotation whose underlying constant ([dcl.attr.annotation]) is r for every reflection $r \in AN_i$
 - (1.3) — has *enumerator-name* N_i , and
 - (1.4) — has an *enumerator-initializer* if and only if $V_i \neq \perp$; if present, it is formed from V_i .
 - (1.4) — The values of enumerators for which $V_i = \perp$ are determined as specified in [dcl.enum] for enumerators without an explicit *enumerator-initializer*.
- Returns:* `targetEnum`.

§ 5. Status

This proposal was implemented ([clang-implementation]) on top of the Clang P2996 branch, and is available for test on the same Compiler Explorer branch.

§ 6. Acknowledgements

The original motivation for this facility was discussed by A. Jiang over [Github issues](#). The early version of this proposal greatly benefitted from Matthias Wippich feedback and insights.

§ References

§ Normative References

[CLANG-IMPLEMENTATION]

Aurelien Cassagnes. *define_enum*. URL: <https://github.com/bloomberg/clang-p2996/pull/263>

[P3294R2]

Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. *Code Injection with Token Sequences*. 15 October 2024. URL: <https://wg21.link/p3294r2>

[P3385R7]

Aurelien Cassagnes. *Attributes reflection*. 16 February 2026. URL: <https://wg21.link/p3385r7>